CyS: TP6
Videla Jerónimo y García Tomás

Trabajo práctico N° 6: Filtrado digital IIR

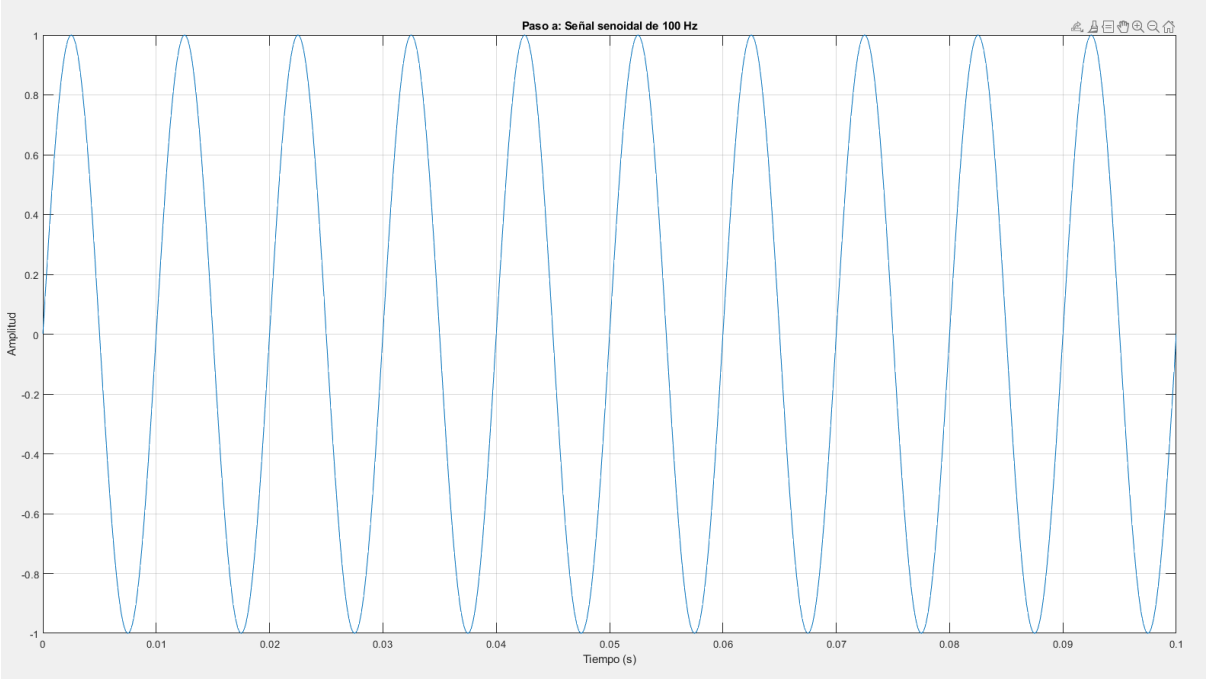
Ejercicio 1:Filtro Leaking Integrator (LI) con señales senoidales en MATLAB

a) Genere una señal senoidal con frecuencia fundamental de 100Hz.

Para comenzar el ejercicio en esta sección se decidió utilizar una frecuencia de muestreo de 10000 Hz usando la técnica de oversampling para lograr una representación más suave en el dominio del tiempo de la señal, y un tiempo tal que permita observar 10 ciclos de la señal. Con esta frecuencia estamos por encima de la frecuencia de muestreo que requiere el teorema de muestreo (200Hz).

```
% --- Inicialización de parámetros ---
fs = 10000;          % Frecuencia de muestreo en Hz
t = 0:1/fs:0.1;      % Vector de tiempo (0.1 segundos)
f0 = 100;            % Frecuencia fundamental en Hz

% a) Genere una señal senoidal con frecuencia fundamental de 100Hz
x = sin(2*pi*f0*t);
```



b) Agregue ruido a la señal senoidal tal que la relación señal a ruido entre la señal senoidal y la señal con ruido sea de 15 dB.

Para agregar la señal de ruido se realizó de dos maneras, ambas dando los mismos resultados. La primera opción es utilizar la función de matlab awgn (Additive White Gaussian Noise). A esta función se le puede pasarla señal original x, el valor de SNR

(Signal Noise Ratio) que se requiere (15), y se le agrega el parámetro 'measured' para que MATLAB calcule automáticamente la potencia de la señal original señal original antes de inyectarle el ruido.

awgn

Add white Gaussian noise to signal

Syntax

```
Y = awgn(X,snr)
Y = awgn(X,snr,signalpower)

Y = awgn(X,snr,signalpower,randobject)
Y = awgn(X,snr,signalpower,seed)
Y = awgn(___,powertype)

[Y,var] = awgn(___)
```

Description

`Y = awgn(X,snr)` adds white Gaussian noise to the vector signal `X`. This syntax assumes that the power of `X` is 0 dBW. For more information about additive white Gaussian noise, see What is AWGN?

`Y = awgn(X,snr,signalpower)` accepts an input signal power value in dBW. To measure the power of `X` before adding noise, specify `signalpower` as `'measured'`. The `'measured'` option does not generate the requested average SNR for repeated `awgn` function calls in a loop if the input signal power varies over time due to fading and the coherence time of the channel is larger than the input duration.

`Y = awgn(X,snr,signalpower,randobject)` additionally accepts a random number stream object to generate normal random noise samples. For information about producing repeatable noise samples, see Tips.

`Y = awgn(X,snr,signalpower,seed)` specifies a seed value for initializing the normal random number generator that is used to add white Gaussian noise to the input signal.

`Y = awgn(___,powertype)` specifies the signal and noise power type as `'dB'` or `'linear'` in addition to the input arguments in any of the previous syntaxes. For information on the relationships between SNR and other measures of the relative power of the noise, such as E_s/N_0 and E_b/N_0, see AWGN Channel Noise Level.

`[Y,var] = awgn(___)` also returns the total noise variance used to produce random noise samples.

```
% b) Agregue ruido a la señal senoidal con SNR = 15 dB
SNR_dB = 15;
x_ruido = awgn(x, SNR_dB, 'measured');
```

La segunda forma en la que se puede agregar la señal de ruido es realizando las acciones paso a paso:

1) Primero se debe calcular la potencia de la señal original. Como la potencia de señales con media nula se puede calcular mediante la varianza, en este caso realizamos eso:

```
potencia_seno = var(x);
```

2) Luego, realizamos el cálculo de la potencia que debe tener la señal de ruido para que la relación señal a ruido entre la señal senoidal y la señal con ruido sea de 15 dB.

```
SNR_dB = 15;
SNR_lineal = 10^(SNR_dB / 10);
potencia_ruido = potencia_seno / SNR_lineal;
```

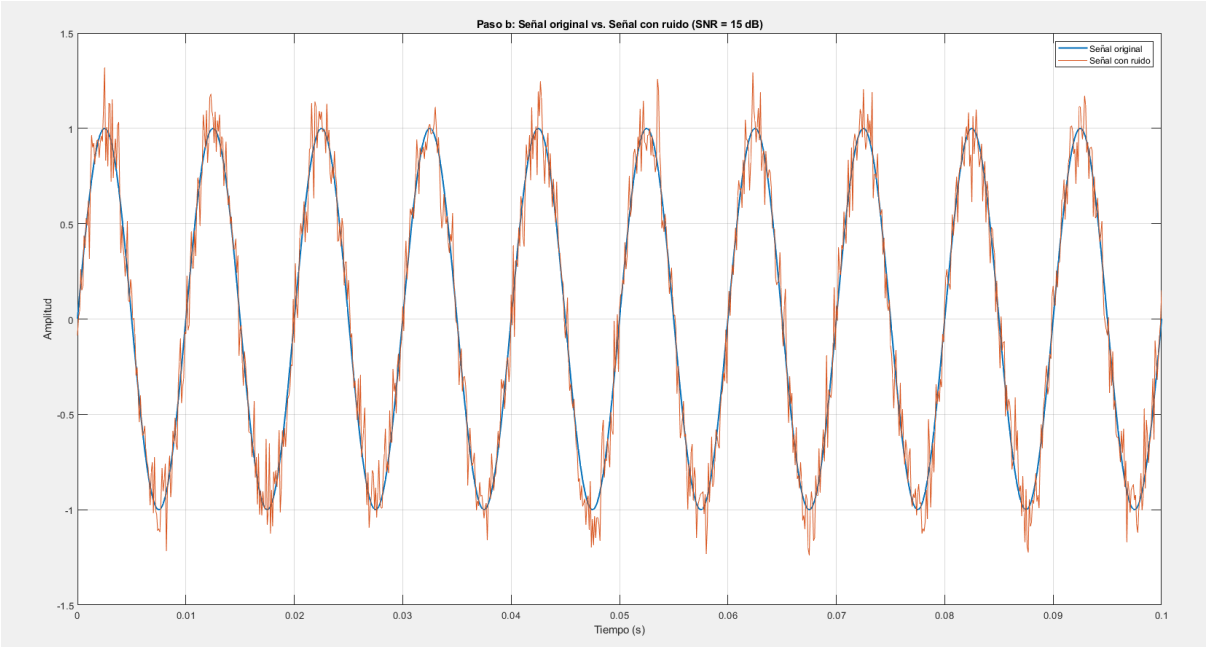
3) Como tercer paso procedemos a generar un vector de ruido blanco gaussiano con la potencia calculada en el paso anterior

```
ruido = sqrt(potencia_ruido) * randn(size(x));
```

4) Como último paso procedemos a sumar ambas ondas, la de ruido y la senoidal

```
x_ruido = x + ruido;
```

De la manera mencionada, obtuvimos la señal original con el ruido debido a la segunda señal generada



c) Diseñe un filtro leaking integrator (LI) con λ igual a 0.7.

Como se ha mencionado en la teoría, el filtro leaking integrator o Filtro integrador con pérdida tiene una función desarrollada:

$$y[n] = \lambda y[n-1] + (1-\lambda)x[n]$$

Que cuando se le aplica la transformada Z y se reordena nos queda una función de transferencia del tipo

$$H(z) = \frac{1-\lambda}{1-\lambda z^{-1}} = \frac{b_0}{a_0 + a_1 z^{-1}}$$

Para armar este filtro en MATLAB, se necesitó definir los vectores de los coeficientes del numerador y denominador. De esa manera, extrayendo los datos directamente de la ecuación nos queda:

```
% c) Diseñe un filtro leaking integrator (LI) con lambda = 0.7
lambda = 0.7;
b = 1 - lambda;
a = [1, -lambda];
```

d) Grafique la respuesta en frecuencia y fase del filtro LI. Use la función freqz().
Determine la frecuencia de corte fco con:

fco = - ln (λ) . fs / π

Como este punto solicita tanto graficar como realizar un cálculo, se comenzará con el desarrollo del gráfico.

Para la parte gráfica,tal como se pide en el enunciado, se utiliza la función freqz de MATLAB, la cual permite que al llamarla sin asignarle variables de salida, se pueda obtener automáticamente una figura con el diagrama de magnitud arriba y el de fase abajo. Debido a que se ha definido la variable fs (frecuencia de muestreo) en el primer paso, le pasamos

dicho valor a la función para que el eje horizontal te quede en Hz, que es mucho más fácil de leer.

```
figure;
freqz(b, a, 1024, fs);
title('Paso d: Respuesta en frecuencia y fase del filtro LI (\lambda = 0.7)');
```

A continuación se puede observar la descripción de la función

freqz

Frequency response of digital filter

Syntax

```
[h,w] = freqz(b,a,n)
[h,w] = freqz(sos,n)
[h,w] = freqz(d,n)
[h,w] = freqz(___,n,'whole')
```

```
[h,f] = freqz(___,n,fs)
[h,f] = freqz(___,n,'whole',fs)
```

```
h = freqz(___,w)
h = freqz(___,f,fs)
```

```
freqz(___)
```

Description

`[h,w] = freqz(b,a,n)` returns the `n`-point frequency response vector `h` and the corresponding angular frequency vector `w` for the digital filter with transfer function coefficients stored in `b` and `a`.

`[h,w] = freqz(sos,n)` returns the `n`-point complex frequency response corresponding to the second-order sections matrix `sos`.

`[h,w] = freqz(d,n)` returns the `n`-point complex frequency response for the digital filter `d`.

`[h,w] = freqz(___,n,'whole')` returns the frequency response at `n` sample points around the entire unit circle.

`[h,f] = freqz(___,n,fs)` returns the frequency response vector `h` and the corresponding physical frequency vector `f` for a digital filter designed to filter signals sampled at a rate `fs`.

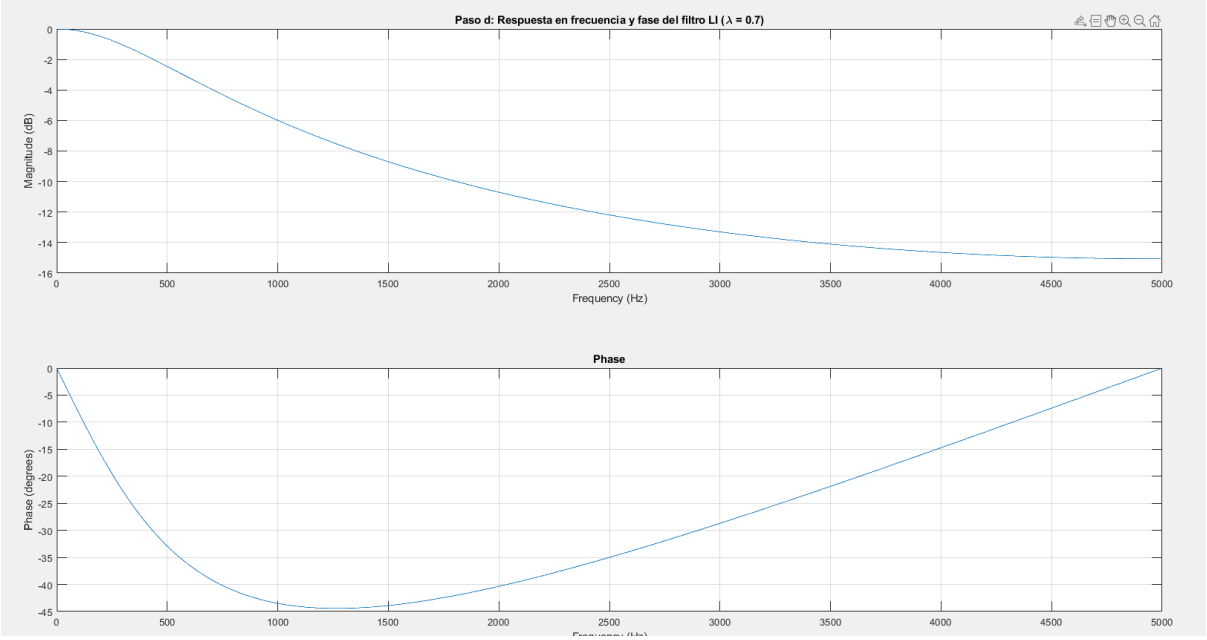
`[h,f] = freqz(___,n,'whole',fs)` returns the frequency vector at `n` points ranging between 0 and `fs`.

`h = freqz(___,w)` returns the frequency response vector `h` evaluated at the normalized frequencies supplied in `w`.

`h = freqz(___,f,fs)` returns the frequency response vector `h` evaluated at the physical frequencies supplied in `f`.

`freqz(___)` with no output arguments plots the frequency response of the filter.

Habiendo hecho uso de la función, el resultado obtenido es:



Para continuar con este paso se realiza el cálculo, el cual una vez realizado en matlab da como resultado:

```
fco = -log(lambda) * fs / pi;
```

```
La frecuencia de corte fco para lambda=0.7 es: 1135.33 Hz
```

e) Determine el cero y el polo del filtro con la función zplane(). ¿Es el filtro estable?.

Como menciona la siguiente imagen de la descripción de la función, la función zplane() dibuja automáticamente el círculo unitario en el plano complejo y marca dónde están ubicados los polos (con una 'x') y los ceros (con una 'o').

zplane

Zero-pole plot for discrete-time systems

coll

Syntax

```
zplane(z,p)
zplane(b,a)
[hz,hp,ht] = zplane( __ )
```

```
zplane(d)
[vz,vp,vk] = zplane(d)
```

Description

`zplane(z,p)` plots the zeros specified in column vector `z` and the poles specified in column vector `p` in the current figure window. The symbol `'o'` represents a zero and the symbol `'x'` represents a pole. The plot includes the unit circle for reference.

If `z` and `p` are matrices, then `zplane` plots the poles and zeros in the columns of `z` and `p` in different colors.

`zplane(b,a)`, where `b` and `a` are row vectors, first uses `roots` to find the zeros and poles of the transfer function represented by the numerator coefficients `b` and the denominator coefficients `a`.

`[hz,hp,ht] = zplane( __ )` returns vectors of handles to the zero lines `hz` and the pole lines `hp`. `ht` is a vector of handles to the axes/unit circle line and to text objects, which are present when there are multiple zeros or poles.

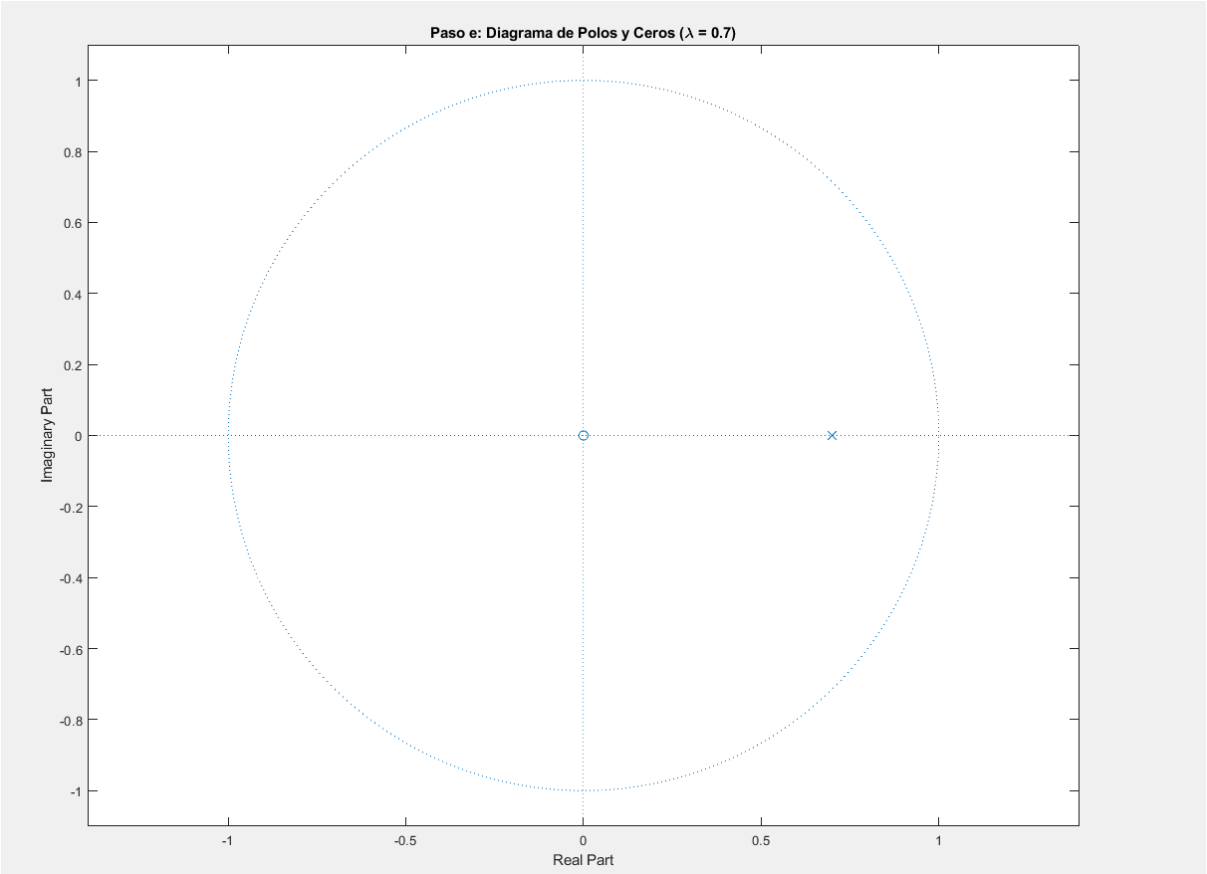
`zplane(d)` finds the zeros and poles of the transfer function represented by the digital filter, `d`. Use `designfilt` to generate `d` based on frequency-response specifications. The pole-zero plot is displayed in FVTool.

`[vz,vp,vk] = zplane(d)` returns the zeros `vz`, poles `vp`, and gain `vk` corresponding to the digital filter `d`.

Gracias a esta función, se puede obtener la imagen que permita observar donde se encuentran los polos y los ceros:

```
figure;
zplane(b, a);
title('Paso e: Diagrama de Polos y Ceros (\lambda = 0.7)');
```

CyS: TP6
Videla Jerónimo y García Tomás



Aunque se obtuvo de manera gráfica, para un manejo más fiable de la información, hacemos uso de la función roots para obtener los polos y ceros del filtro.

```
ceros = roots(b);
polos = roots(a);

fprintf('\n Polos y ceros del filtro\n');
disp('Ceros del filtro (raíces del numerador):');
disp(ceros);
disp('Polos del filtro (raíces del denominador):');
disp(polos);
```

```
 Polos y ceros del filtro
Ceros del filtro (raíces del numerador):
Polos del filtro (raíces del denominador):
    0.7000
```

roots

Polynomial roots

Syntax

```
r = roots(p)
```

Description

`r = roots(p)` returns the roots of the polynomial represented by `p` as a column vector. Input `p` is a vector containing `n+1` polynomial coefficients, starting with the coefficient of x^n. For example, `p = [3 2 -2]` represents the polynomial $3x^2 + 2x - 2$. A coefficient of `0` indicates an intermediate power that is not present in the equation.

The `roots` function solves polynomial equations of the form $p_1x^n + ... + p_nx + p_{n+1} = 0$. Polynomial equations contain a single variable with nonnegative exponents.

Como se puede observar, la función roots obtiene las raíces del numerador y denominador de la función de transferencia, y de esta manera obteniendo las raíces del numerador conseguimos los ceros, y con las raíces del denominador los polos

Lo único que queda por definir en este caso es la pregunta del enunciado: ¿Es el filtro estable?

Al analizar la función de transferencia del filtro, observamos que posee un único polo real ubicado en $z = \lambda = 0.7$. La teoría de sistemas discretos establece que un filtro es estable si y sólo si todos sus polos se encuentran estrictamente dentro del círculo unitario. En el caso del filtro Leaking Integrator, la condición de estabilidad es $|\lambda| < 1$. Como en este diseño $\lambda = 0.7$, el polo se ubica dentro del círculo unitario, por lo que podemos afirmar que el filtro es estable.

f) Aplique el filtro LI a la señal con ruido. Utilice la función filter().

La función filter() toma los coeficientes del filtro (b y a) que definimos en el paso c, y procesa la señal de entrada paso a paso resolviendo la ecuación en diferencias internamente. A continuación se puede observar la documentación de matlab respecto a la función:

filter

1-D digital filter

Syntax

```
y = filter(b,a,x)
y = filter(b,a,x,zi)
y = filter(b,a,x,zi,dim)
[y,zf] = filter( __ )
```

Description

`y = filter(b,a,x)` filters the input data `x` using a rational transfer function defined by the numerator and denominator coefficients `b` and `a`.

If `a(1)` is not equal to `1`, then `filter` normalizes the filter coefficients by `a(1)`. Therefore, `a(1)` must be nonzero.

- If `x` is a vector, then `filter` returns the filtered data as a vector of the same size as `x`.
- If `x` is a matrix, then `filter` acts along the first dimension and returns the filtered data for each column.
- If `x` is a multidimensional array, then `filter` acts along the first array dimension whose size does not equal 1.

`y = filter(b,a,x,zi)` uses initial conditions `zi` for the filter delays. The length of `zi` must equal `max(length(a),length(b))-1`.

`y = filter(b,a,x,zi,dim)` acts along dimension `dim`. For example, if `x` is a matrix, then `filter(b,a,x,zi,2)` returns the filtered data for each row.

`[y,zf] = filter( __ )` also returns the final conditions `zf` of the filter delays, using any of the previous syntaxes.

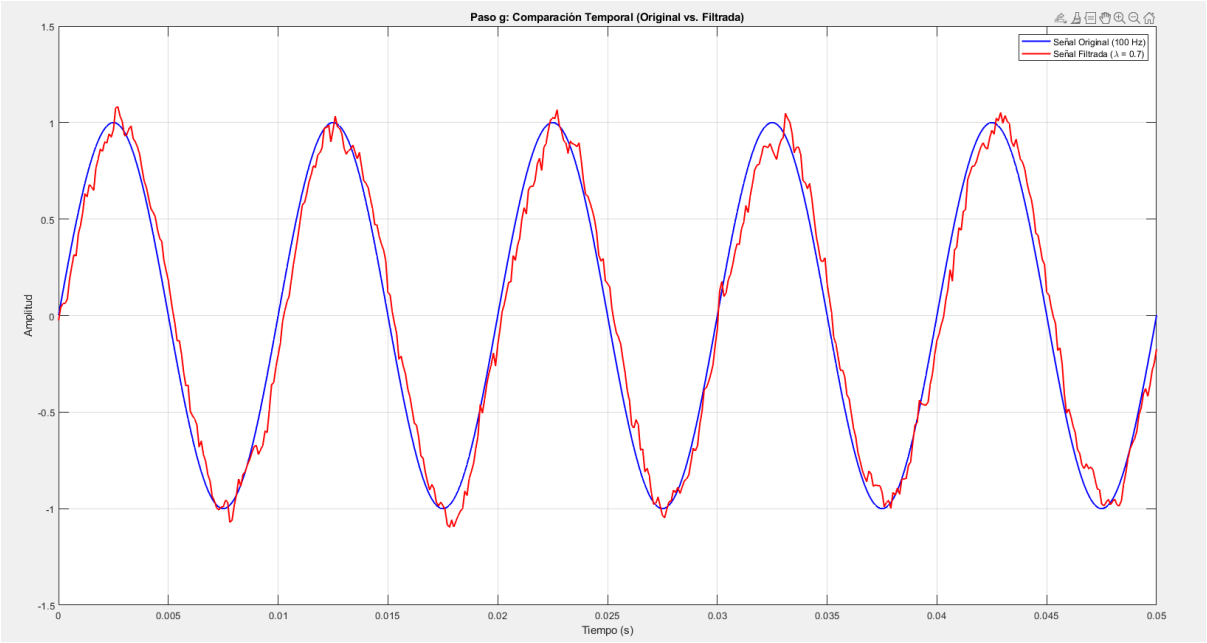
De esta forma, con la linea observada a continuación, queda aplicado el filtro a la función con ruido:

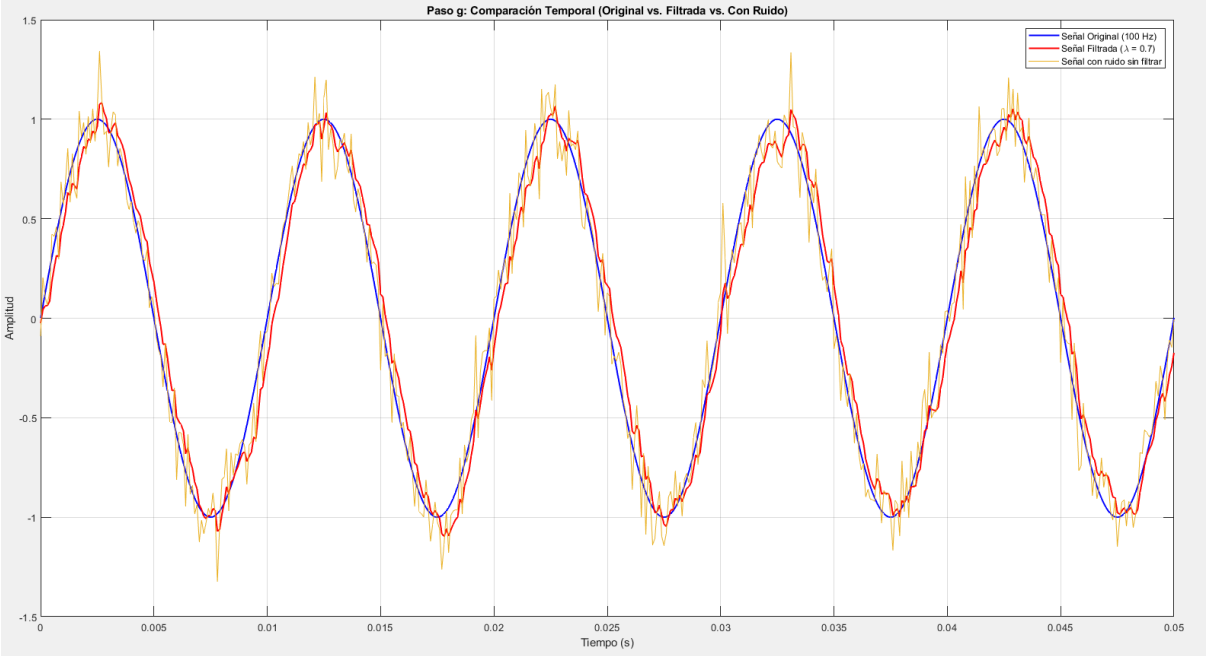
```
x_filtrada = filter(b, a, x_ruido);
```

g) Grafique la respuesta en el tiempo de las señales original y filtrada y compare.

Para obtener una buena comparación se realizó la gráfica en un tiempo limitado a 0.05 segundos, permitiendo ver con mayor claridad cada una de las ondas.

Se realizaron dos imágenes, en la primera se logra comparar la onda filtrada con la original sin ruido, mientras que en la segunda se pueden observar las 3 ondas, la que no posee ruido, la ruidosa sin filtrar, y la filtrada.





Comparación en el dominio del tiempo:

Al observar la señal filtrada en comparación con la original, se destacan los siguientes efectos del filtro Leaking Integrator ($\lambda = 0.7$):

1. Reducción de ruido: La señal filtrada presenta un contorno mucho más suave en comparación con la señal ruidosa de entrada, evidenciando la atenuación de las componentes de alta frecuencia correspondientes al ruido blanco.
2. Desplazamiento de fase (Retardo): Se observa un ligero corrimiento hacia la derecha de la señal filtrada respecto a la original. Este retardo temporal es una consecuencia directa del procesamiento causal de la ecuación en diferencias y la fase no lineal del filtro IIR.

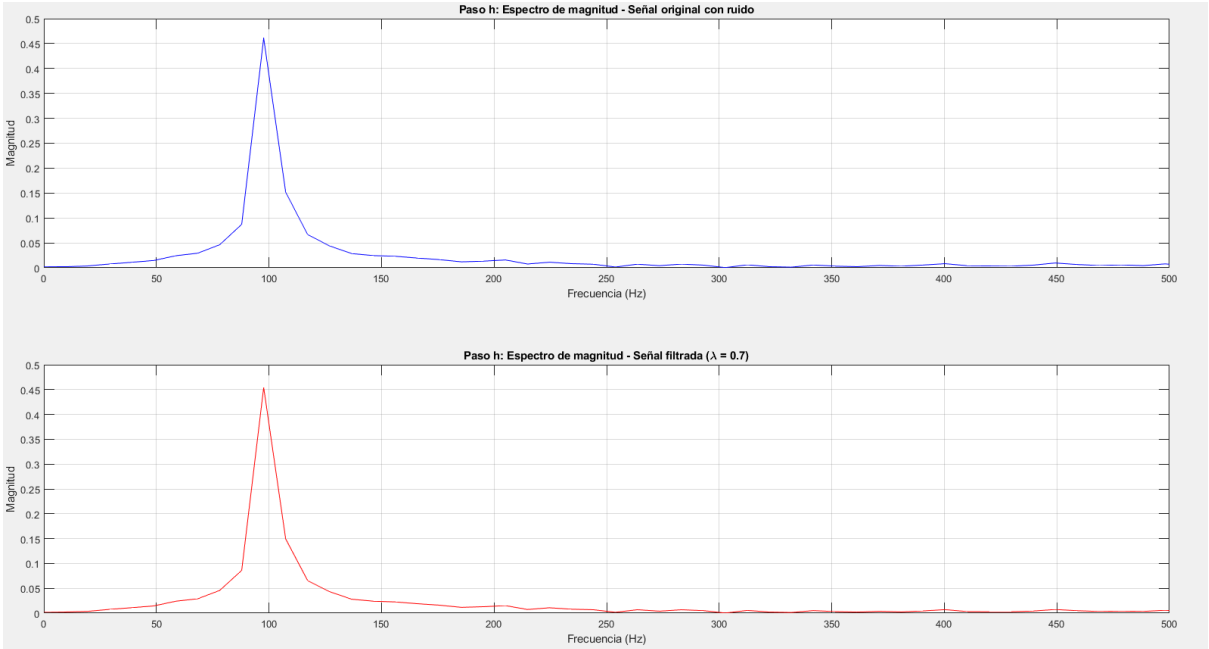
3. Amplitud: La amplitud de la señal fundamental (100 Hz) se mantiene bastante cercana a la original, indicando que esta frecuencia se encuentra dentro de la banda pasante del filtro.

h) Grafique la respuesta en frecuencia de las señales original y filtrada y compare. Utilice la función provista my_dft().

Como se puede ver en la información de la función de la siguiente imagen, la función realiza la matemática de la Transformada Discreta de Fourier y devuelve los vectores calculados. Te entrega el vector de frecuencias (f), la magnitud (dft_mag) y la fase (dft_phase), etc.

```
function [f, dft_mag, dft_phase, dft, NFFT] = my_dft(data, Fs)
% my_dft: devuelve la transformada discreta de Fourier de una serie de tiempo.
%
% Si la unidad de data es U, la unidad de la TDF entregada es U/Hz.
%
% INPUT
%   data: NxD serie de tiempo.
%   Fs: 1x1 frecuencia de muestreo de data.
%
% OUTPUT
%   f         : Nx1 vector frecuencia, en Hz.
%   dft_mag   : NxD magnitud de la transformada discreta de Fourier.
%   dft_phase : NxD fase de la transformada discreta de Fourier.
%   dft       : NxD transformada de Fourier discreta, valores complejos.
%   NFFT      : 1x1 cantidad de elementos usados en la FFT.
```

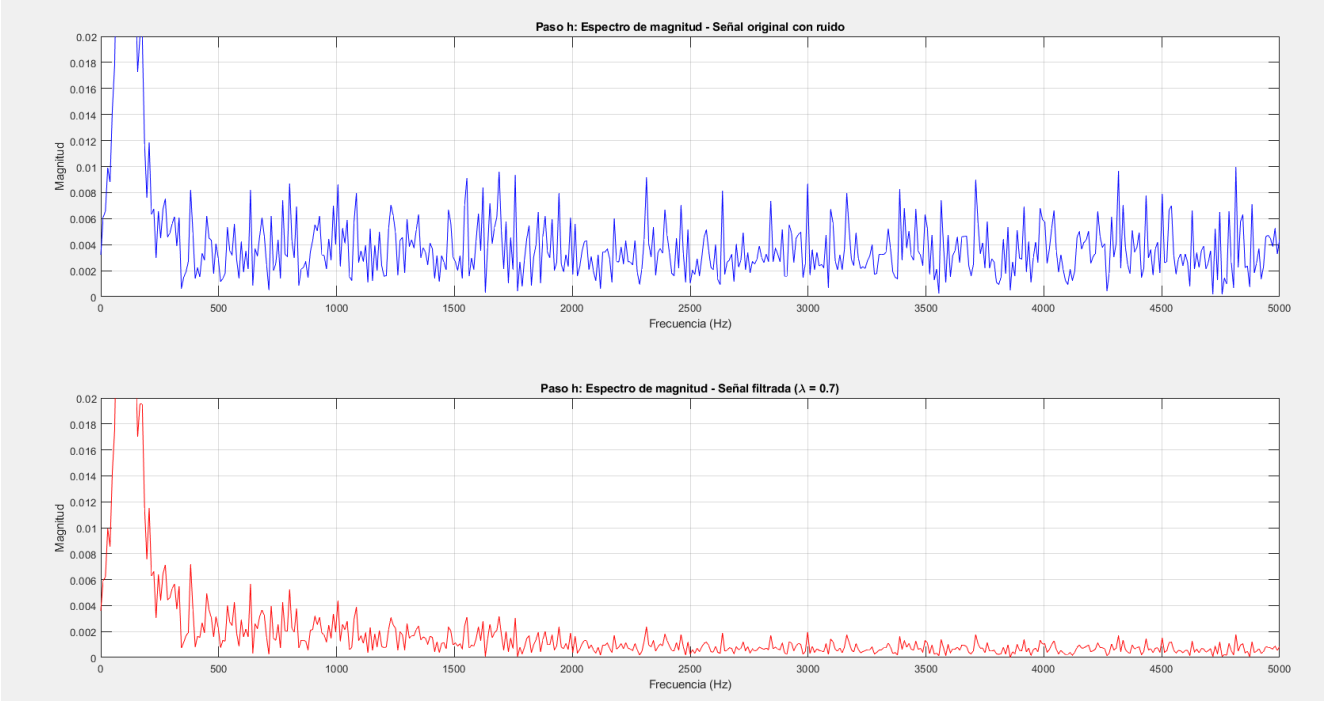
De esta manera, se debe llamar a la función, guardar los resultados, y realizar un ploteo según lo que se obtuvo.



De esta manera, se puede observar que en la gráfica azul hay un pico en 100Hz y muchos de valores con menor magnitud (ruido) en las otras frecuencias. En la gráfica roja,

el pico en 100 Hz sigue firme, pero el ruido a medida que aumentan los valores de la frecuencia se debería atenuar la amplitud debido al efecto del filtro pasabajo. (Como el máximo que se observa es 500 y la frecuencia de corte es de 1135.33Hz ambas gráficas se ven iguales)

En la siguiente imagen se puede observar la misma gráfica, solo que con la totalidad de los valores en las abscisas y con un máximo reducido en las ordenadas para poder observar con mayor precisión el funcionamiento de este filtro.



i) Repita los puntos c) a h) para λ igual a 0.9 y 0.98. Analice el comportamiento de la fco.

Ahora se podrá observar cómo se repiten los diferentes ejercicios con una lambda diferente, pero en este caso no se realizará una explicación de todo el proceso sino que se mostrarán los resultados y conclusiones.

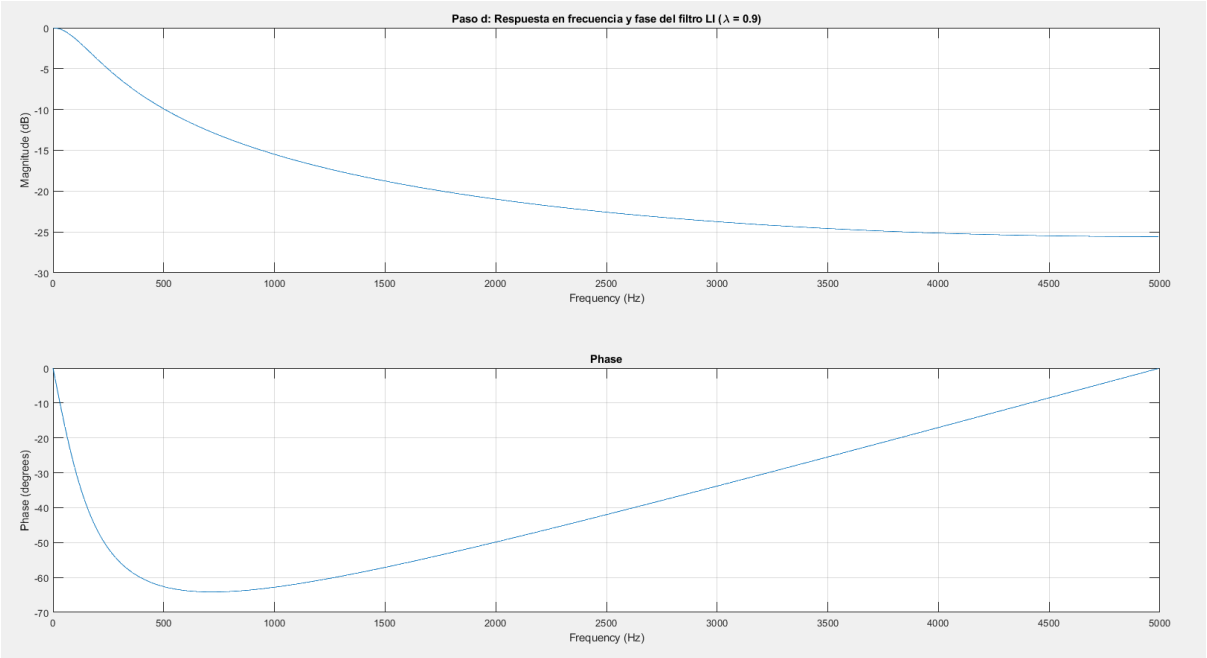
λ igual a 0.9

j) Diseñe un filtro leaking integrator (LI) con λ igual a 0.9.

```
lambda = 0.9;
b = 1 - lambda;
a = [1, -lambda];
```

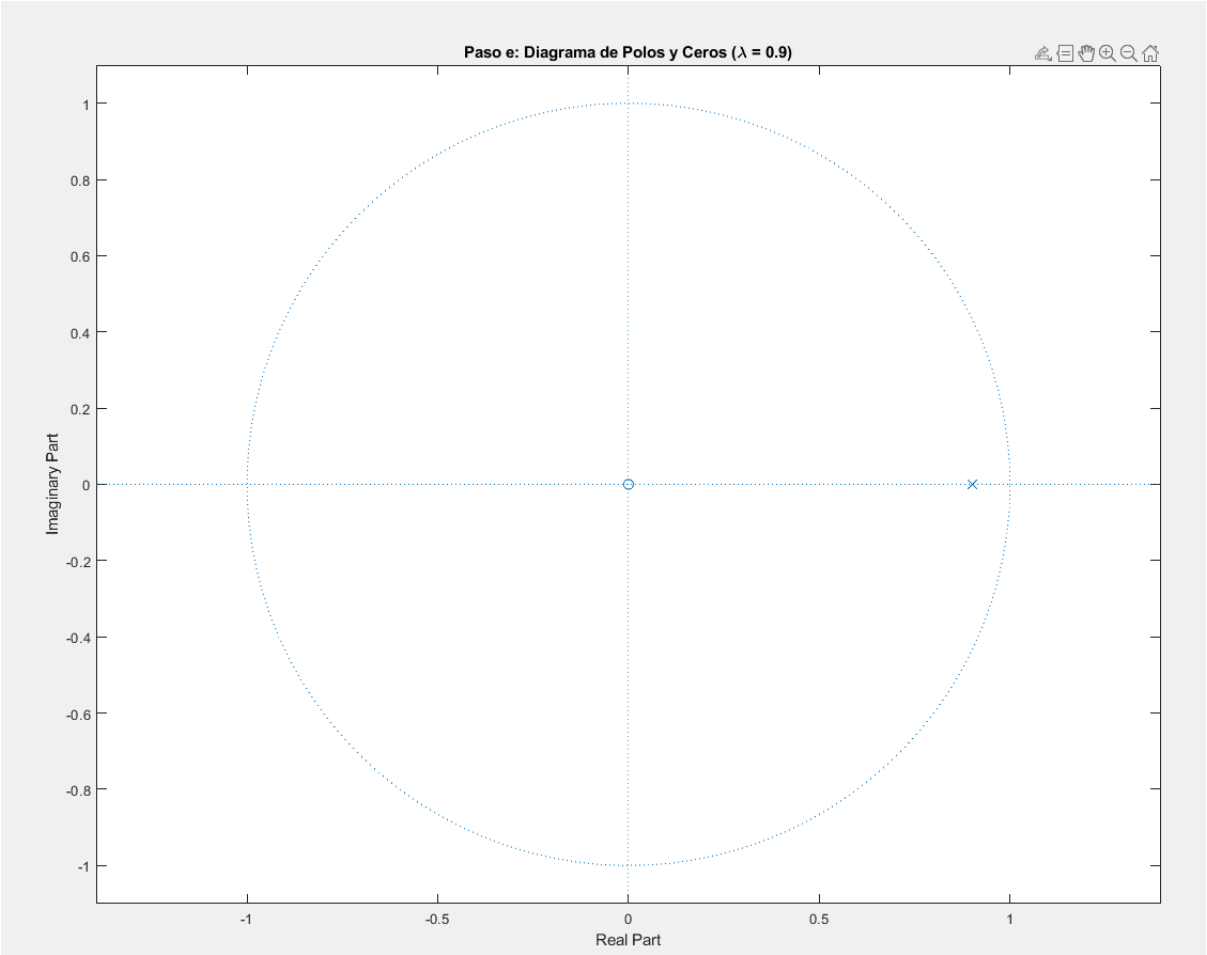
k) Grafique la respuesta en frecuencia y fase del filtro LI. Use la función freqz(). Determine la frecuencia de corte.

```
La frecuencia de corte fco para lambda=0.9 es: 335.37 Hz
```



l) Determine el cero y el polo del filtro con la función zplane(). ¿Es el filtro estable?.

```
 Polos y ceros del filtro
Ceros del filtro (raíces del numerador):
Polos del filtro (raíces del denominador):
    0.9000
```

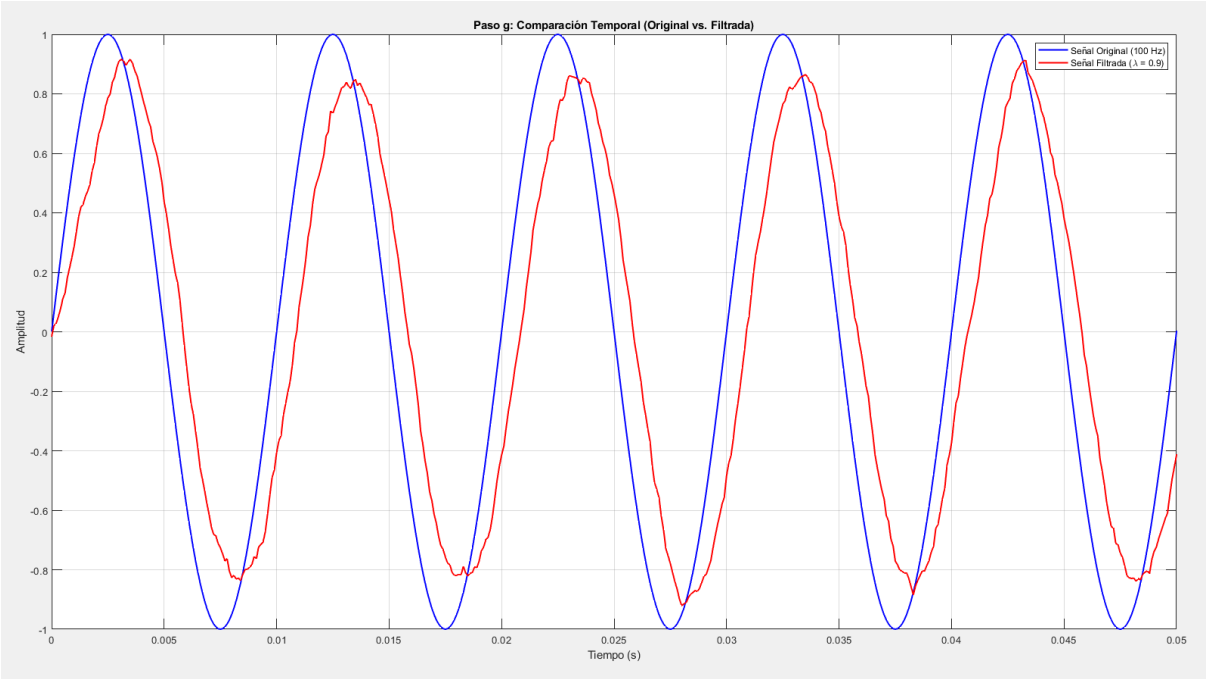


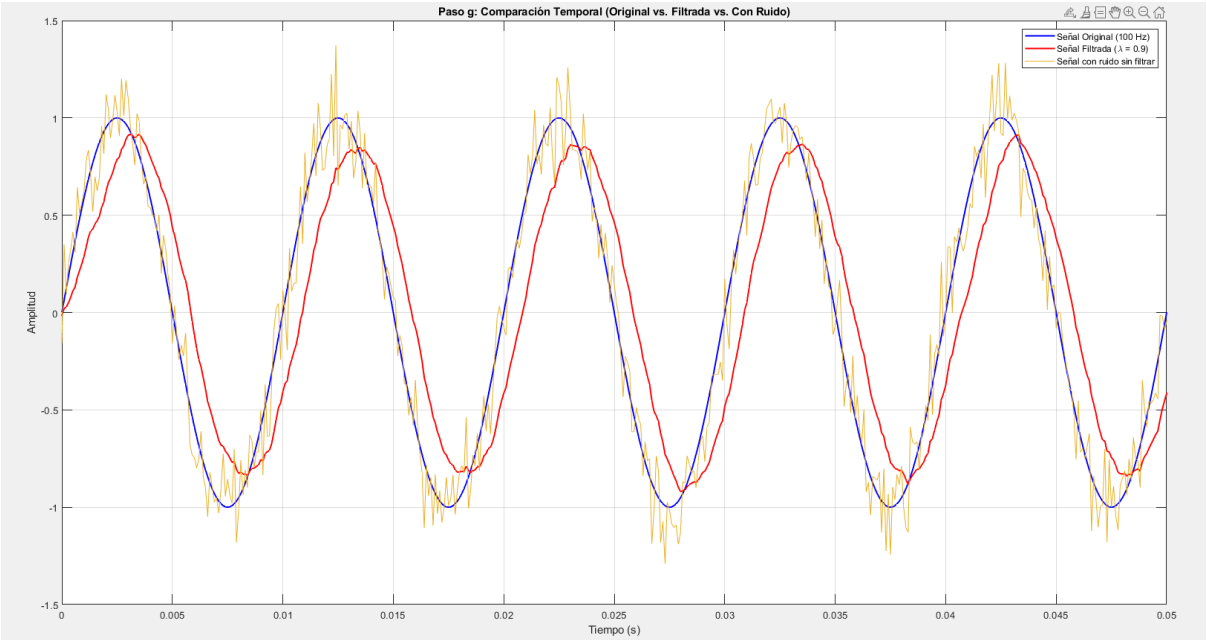
Como el valor del polo es menor a 1 el filtro es estable

m) Aplique el filtro LI a la señal con ruido. Utilice la función filter().

```
x_filtrada = filter(b, a, x_ruido);
```

n) Grafique la respuesta en el tiempo de las señales original y filtrada y compare.

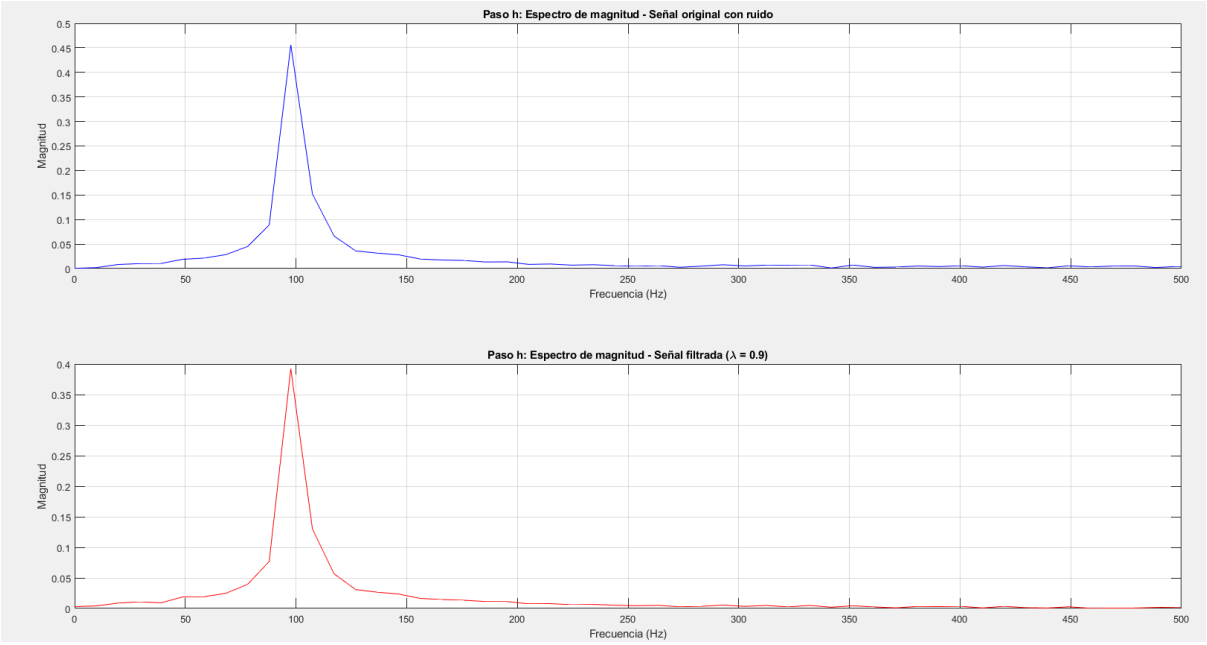


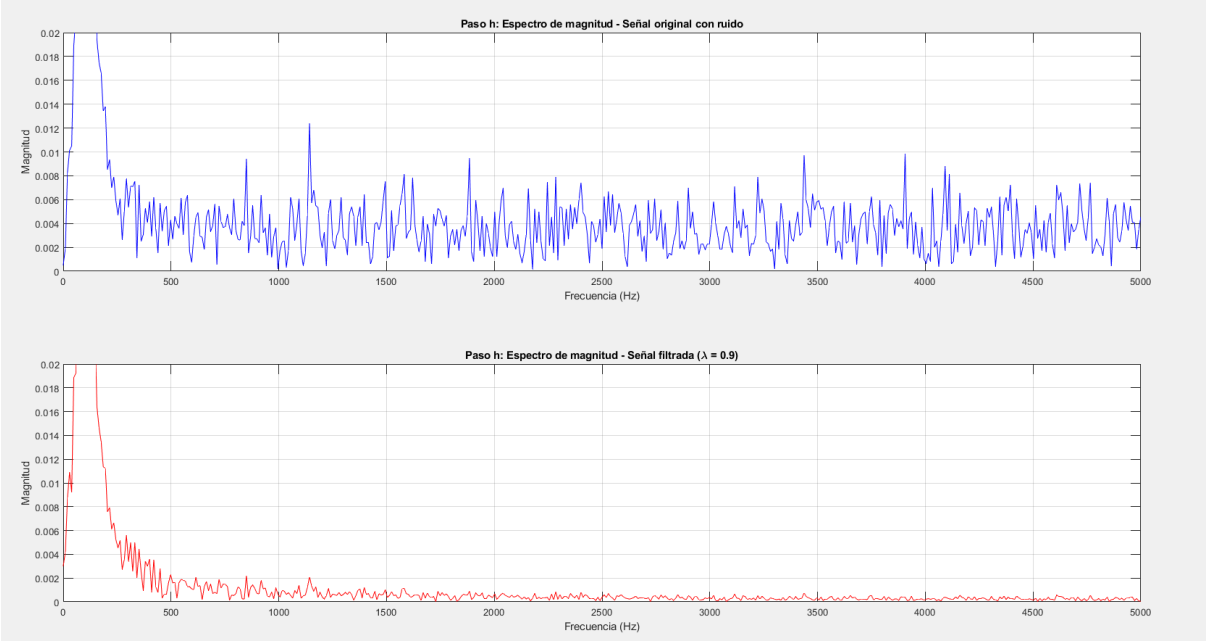


Atenuación del ruido: La señal filtrada (roja) logra seguir la forma de la onda original, pero todavía presenta un contorno visiblemente "dentado" o rugoso. Esto indica que la frecuencia de corte es lo suficientemente alta como para dejar pasar componentes de ruido de alta frecuencia.

Amplitud y Fase: La amplitud de la señal filtrada se mantiene bastante alta (cercana a 0.9) y el desfasaje (retardo temporal) respecto a la original es pequeño. La señal fundamental de 100 Hz no sufre grandes alteraciones.

o) Grafique la respuesta en frecuencia de las señales original y filtrada y compare. Utilice la función provista my_dft().





Banda pasante: Nuestra señal de interés (100 Hz) es menor a la frecuencia de corte ($100 < 335$), por lo que cae de lleno en la banda pasante del filtro.

Efecto en el espectro: En la gráfica vemos que la energía del pico fundamental en 100 Hz se conserva intacta. Por otro lado, el piso de ruido de alta frecuencia (las oscilaciones por encima de los 300-400 Hz) se ve claramente "aplastado" en la gráfica roja. Sin embargo, el ruido de baja frecuencia logra pasar, lo que explica por qué la onda temporal aún se veía un poco dentada.

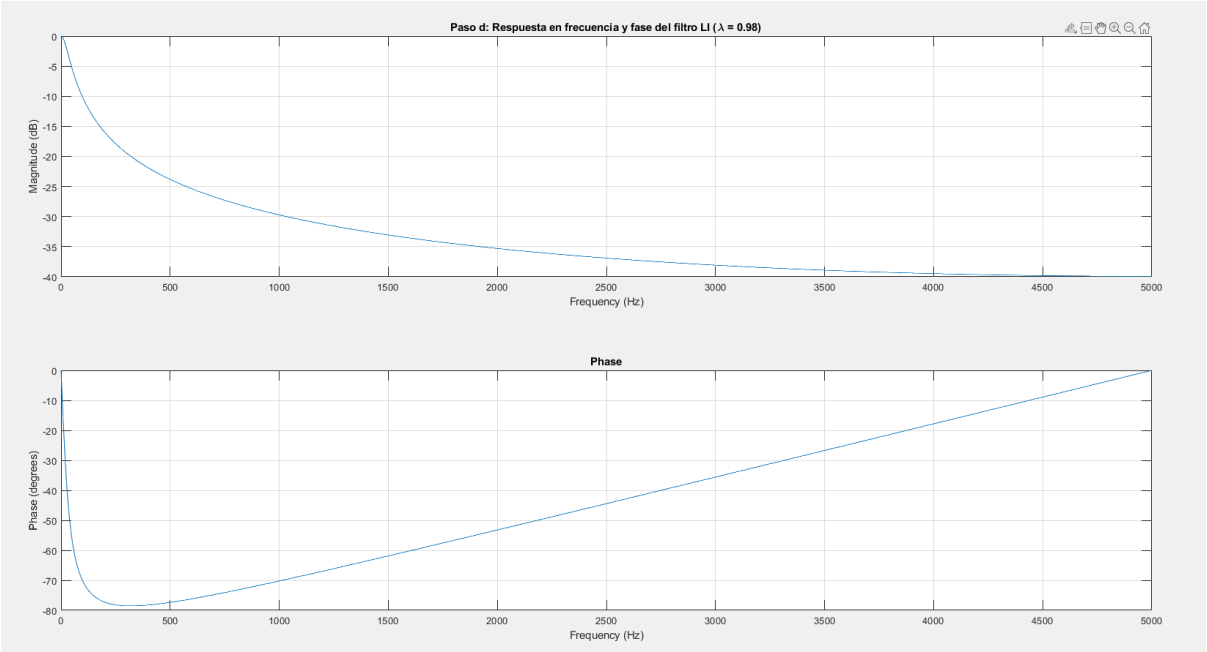
λ igual a 0.98

p) Diseñe un filtro leaking integrator (LI) con λ igual a 0.9.

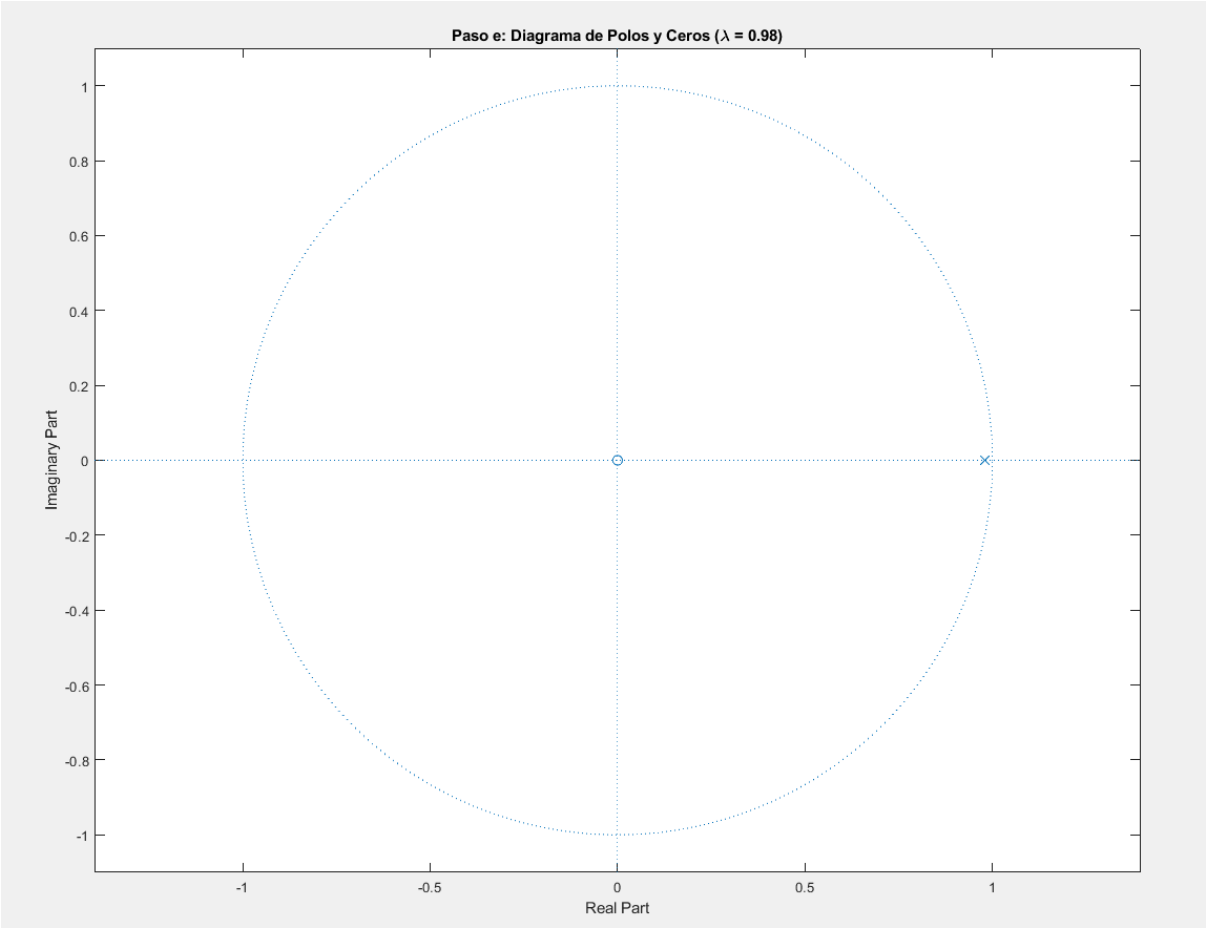
```
lambda = 0.98;
b = 1 - lambda;
a = [1, -lambda];
```

q) Grafique la respuesta en frecuencia y fase del filtro LI. Use la función freqz(). Determine la frecuencia de corte.

```
La frecuencia de corte fco para lambda=0.98 es: 64.31 Hz
```



r) Determine el cero y el polo del filtro con la función zplane(). ¿Es el filtro estable?.

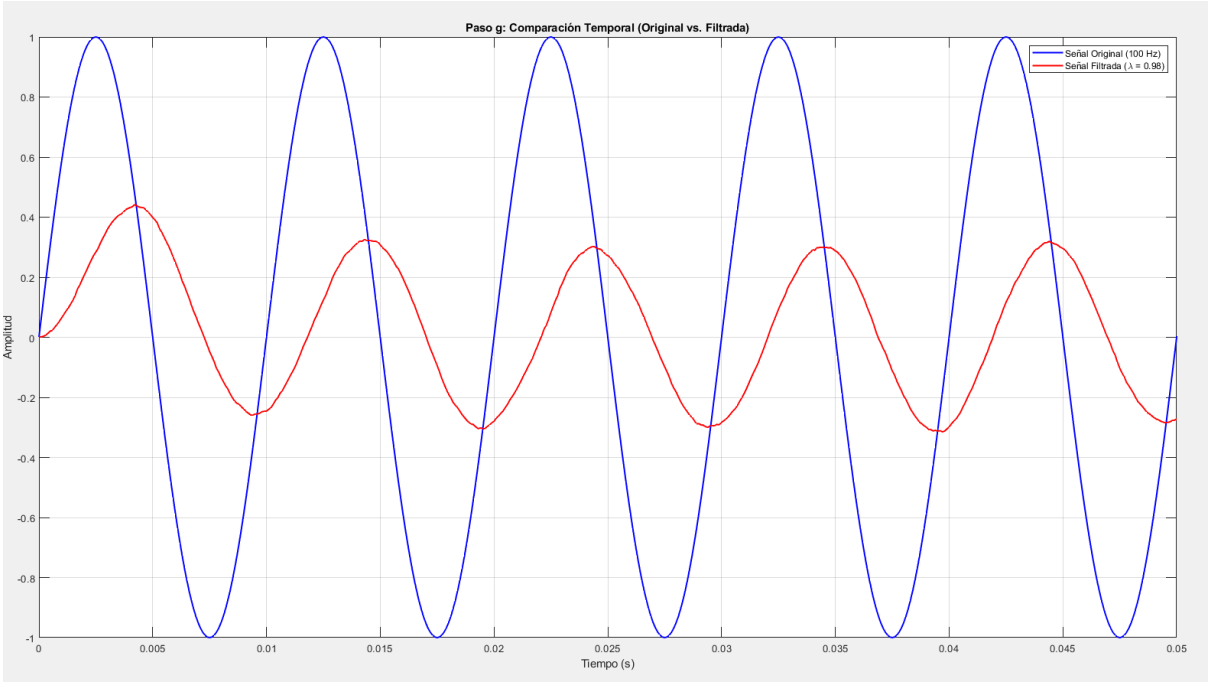


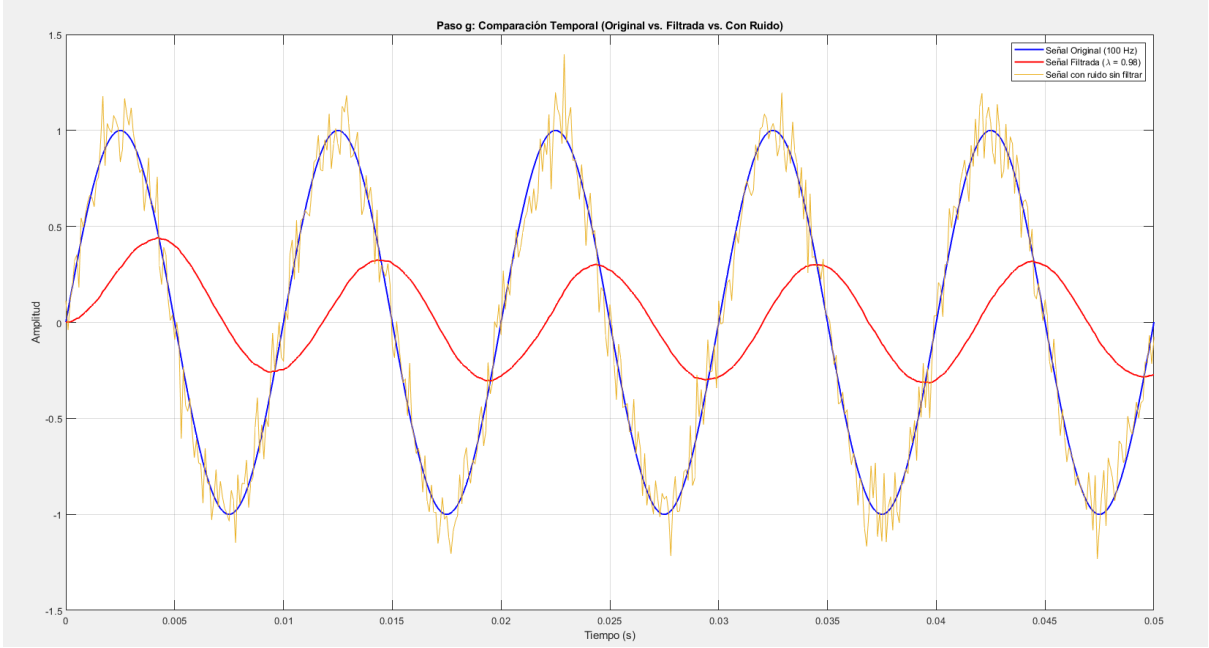
Como el valor del polo es menor a 1 el filtro es estable

s) Aplique el filtro LI a la señal con ruido. Utilice la función filter().

```
x_filtrada = filter(b, a, x_ruido);
```

t) Grafique la respuesta en el tiempo de las señales original y filtrada y compare.

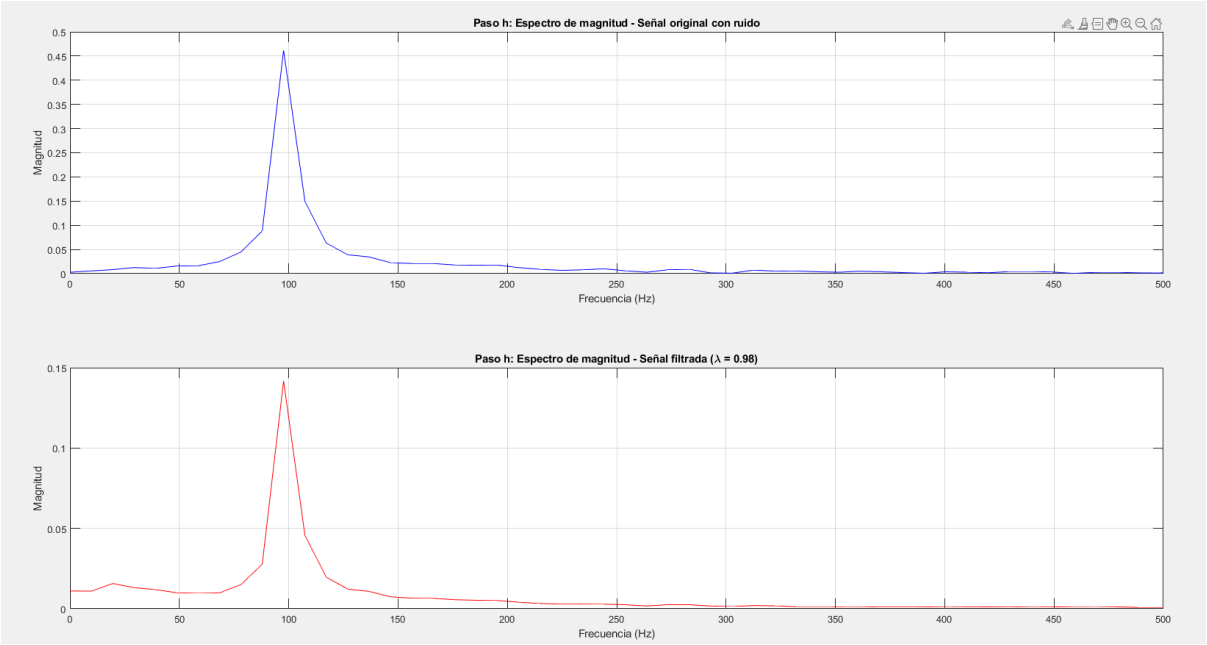


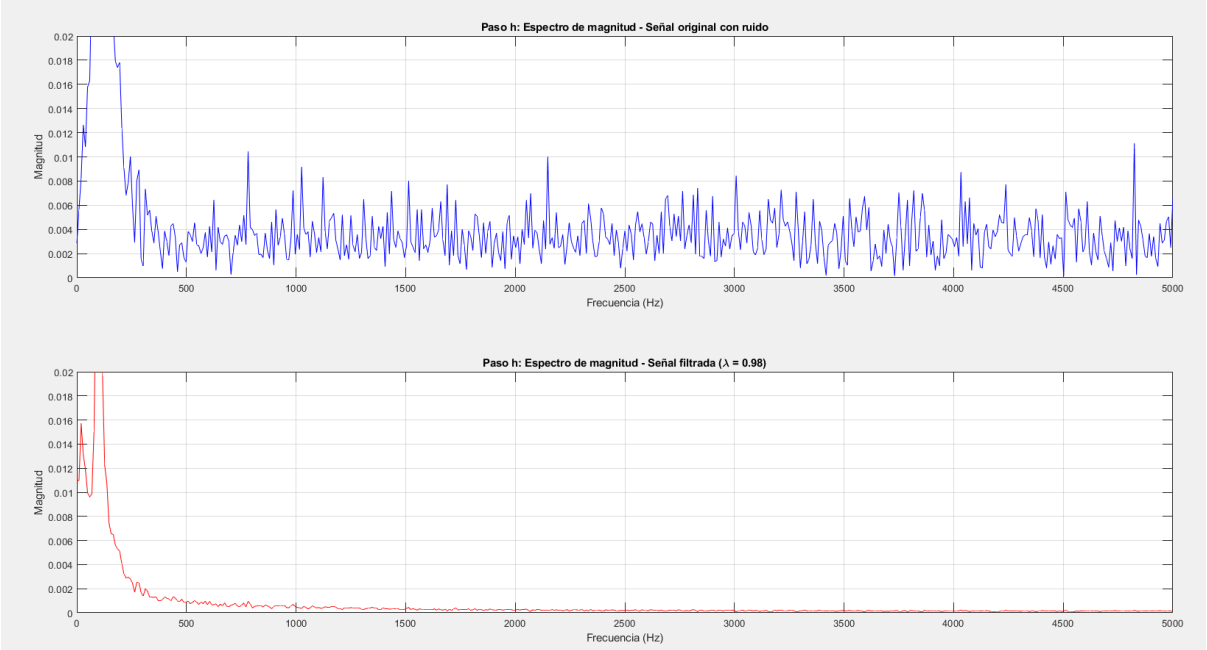


Atenuación del ruido: El filtrado del ruido es excelente. La señal filtrada es una curva completamente suave, lo que demuestra que la banda pasante se ha estrechado drásticamente, eliminando casi por completo las frecuencias altas indeseadas.

Amplitud y Fase: Como consecuencia de una frecuencia de corte mucho más baja, la señal de 100 Hz sufre una fuerte atenuación (su amplitud máxima cae a un valor cercano a 0.3 o 0.4). Además, se observa un desfasaje muy pronunciado (retardo notorio de los picos rojos frente a los azules) y un régimen transitorio más largo al inicio de la simulación, característico de polos muy cercanos al círculo unitario ($z = 1$).

u) Grafique la respuesta en frecuencia de las señales original y filtrada y compare. Utilice la función provista my_dft().

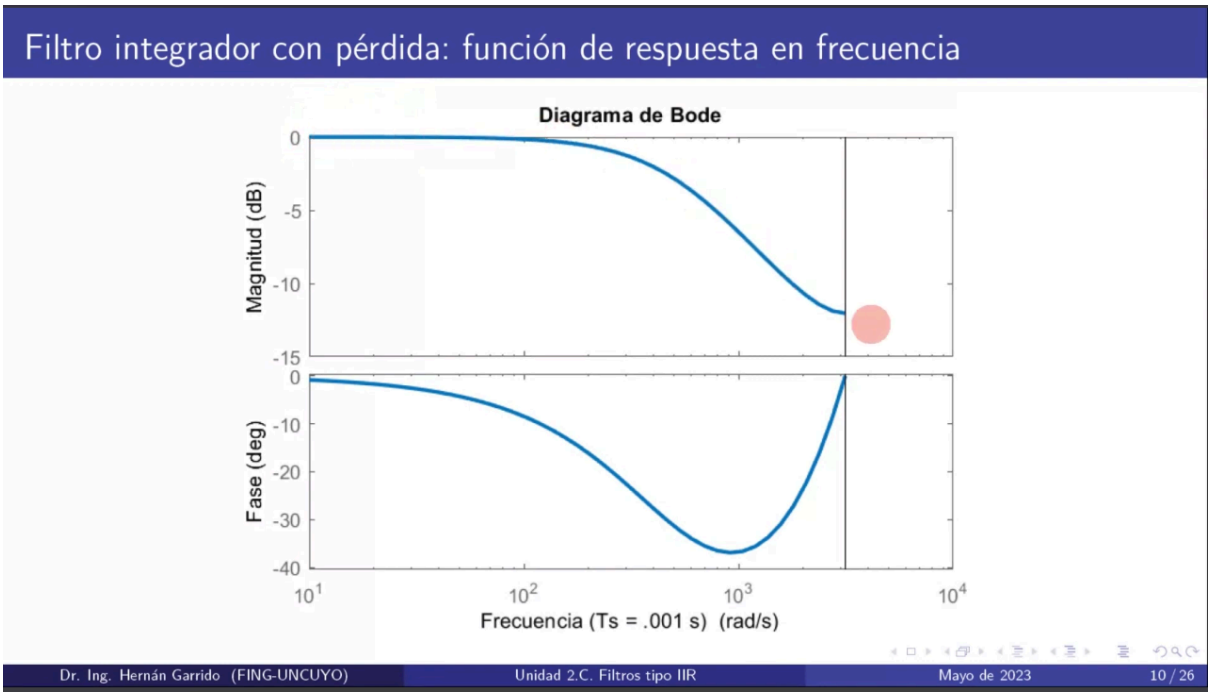




Atenuación de la fundamental: Acá ocurre el fenómeno más crítico. La frecuencia de corte cayó por debajo de nuestra señal fundamental ($100 > 64.31$). Por lo tanto, la señal de 100 Hz quedó fuera de la banda pasante y entró en la zona de atenuación progresiva del filtro.

Efecto en el espectro: En la gráfica roja se observa que el filtro "planchó" el espectro casi por completo. El piso de ruido blanco desapareció en las frecuencias altas (filtrado excelente). Sin embargo, al haber estrechado tanto la campana del filtro, también se redujo drásticamente la magnitud de nuestra señal útil de 100 Hz (lo cual se corrobora con la pérdida de amplitud que habíamos notado en el dominio del tiempo).

¿Qué es lo que pasa matemática y físicamente cuando se aumenta el valor de λ ? :

CyS: TP6
Videla Jerónimo y García Tomás

1. ¿Qué pasa con el valor numérico?
   A medida que aumenta λ (0.7 $\rightarrow$ 0.9 $\rightarrow$ 0.98), se puede observar que la frecuencia de corte (f_{co}) se hace cada vez más pequeña.
   - La fórmula es $f_{co} = \dfrac{-\ln(\lambda) \cdot f_s}{\pi}$. Como el logaritmo natural de un número que tiende a 1 tiende a 0, la f_{co} disminuye drásticamente.
2. ¿Qué significa esto en el filtro?
   Al achicarse la frecuencia de corte, la banda pasante del filtro se hace mucho más "estrecha" (más fina). El filtro se vuelve más restrictivo.
3. ¿Cómo se ve en las gráficas temporales y de frecuencia?
   - Lo bueno: Se puede notar que con $\lambda = 0.98$ la señal filtrada queda muchísimo más suave y limpia de ruido que con 0.7, debido a que se corta casi toda la "basura" de alta frecuencia.
   - Lo malo: Como el polo se acerca al borde del círculo unitario ($z = 1$), el filtro se vuelve más "lento" para reaccionar. Se puede observar en el gráfico temporal que la señal filtrada tarda unos milisegundos más en arrancar (mayor transitorio) y tiene un poco más de retardo (desfasaje) respecto a la onda original.